GUÍA MAESTRA PARA DEFENSA DE GRADO

Sistema Web Xiorizos Salón & Studio

Documento técnico estructurado para preparación y defensa oral de tesis • Arquitectura Full-Stack, Base de Datos, Flujo Operativo e Inteligencia Artificial en el Cliente

1. Resumen Ejecutivo (Discurso de Apertura - 1 Minuto)

> *"Estimado jurado evaluador, el proyecto **Xiorizos** es una plataforma integral desarrollada para modernizar la gestión comercial y operativa de un salón especializado en cabello rizado en Valencia, Venezuela. La plataforma resuelve la captación y fidelización de clientas a través de tres pilares: un comercio electrónico y sistema de citas adaptado a la economía nacional con Pago Móvil interbancario y cobro de reserva del 20%; un panel administrativo integral con control de agendas e inoperatividad médica; y un simulador de colorimetría capilar basado en Inteligencia Artificial y Visión Artificial que se ejecuta al 100% en el dispositivo del cliente mediante WebAssembly, garantizando cero costo de servidores y absoluta privacidad de las fotos."*

2. Arquitectura Global del Sistema

El sistema implementa una arquitectura moderna de **tres capas desacopladas** sobre un entorno monolítico ágil con **Next.js 14 App Router**:

Capa	Componentes Clave	Función Principal
1. Frontend (Cliente)	React 18, Tailwind CSS, Zustand, MediaPipe WASM.	Renderizado reactivo sin parpadeo (SPA Híbrida), gestión de carrito global en memoria y procesamiento local de IA.
2. Backend (Servidor)	Next.js Route Handlers (`/api/*`), Middleware RBAC.	Validaciones de reglas de negocio, prevención de colisiones en agenda, control de acceso por roles y cálculos contables.
3. Persistencia (Datos)	Archivo transaccional `database.json` con caché en memoria.	Almacenamiento atómico de usuarios, citas, órdenes, productos y personal, protegido contra escrituras concurrentes.

3. Tecnologías Seleccionadas y Justificación Técnica

Tecnología	Rol en el Proyecto	¿Por qué se eligió? (Defensa de Tesis)
Next.js 14 (App Router)	Framework Full-Stack	Unifica servidor y cliente en TypeScript. Ofrece **Server-Side Rendering (SSR)** para indexación y **Client-Side Rendering (CSR)** para navegación ultrarrápida sin recargas completas.
TypeScript 5	Lenguaje de Programación	Tipado estático en tiempo de desarrollo. Previene errores en producción al asegurar contratos de datos estrictos en citas y pedidos.
Tailwind CSS + Lucide	Diseño UI & Iconografía	Estilizado atómico sin dependencias pesadas. Asegura una experiencia 100% responsiva (móviles, tablets y pantallas de escritorio).
Zustand	Gestión de Estado Global	Pesa menos de 1 KB (frente a los más de 40 KB de Redux). Sincroniza en tiempo real el carrito de compras y la sesión sin llamadas redundantes al servidor.
Google MediaPipe (WASM)	Inteligencia Artificial Local	Ejecución de redes neuronales convolucionales en el navegador mediante **Edge Computing**. **$0 costo en API** y privacidad total para la clienta.
Node.js + Sharp	Backend y Motor de Imágenes	Optimización y redimensionamiento dinámico de fotos subidas por administradores a formato WebP moderno de alta eficiencia.

4. ¿Cómo Funciona la Base de Datos y Dónde se Guarda la Información?

> **Ubicación Física de la Base de Datos:** Archivo `data/database.json` en la raíz del servidor.
> **Módulo de Acceso Centralizado:** `src/lib/db/index.ts` con bloqueo de lectura y escrituras atómicas (`fs.renameSync`).

La base de datos almacena 5 colecciones esenciales para la operación del salón:

1. **Usuarios (`db.users`):** Almacena perfiles con ID único, nombre, correo, teléfono, rol (`admin`, `empleado`, `cliente`) y contraseña con validación de seguridad (mínimo 8 caracteres, mayúscula, minúscula, número y símbolo).
2. **Citas y Agenda (`db.appointments`):** Almacena cada cita con cliente, servicio, estilista asignado, fecha, hora, notas capilares, estado (`pendiente`, `confirmada`, `completada`, `cancelada`) y el desglose financiero:
   - **Monto Total:** Costo total del servicio en salón.
   - **Monto de Reserva (20%):** Abono obligatorio adelantado para apartar el cupo.
   - **Monto Restante (80%):** Saldo a cancelar presencialmente en el estudio.
   - **Datos de Pago Móvil:** Banco emisor, número de referencia bancaria y teléfono emisor.
3. **Pedidos de Productos (`db.orders`):** Código de orden (`ORD-2026-XXXX`), cliente, dirección en Valencia/Naguanagua, lista de productos comprados, método de pago y estado logístico.
4. **Productos e Inventario (`db.products`):** Catálogo botánico con nombre, descripción, precio en USD ($), costo unitario, stock en tiempo real y valoración de clientes.
5. **Personal y Control de Inoperatividad (`db.employees`):** Registro de estilistas con servicios asignados y módulo de inasistencias: permite programar reposos médicos o vacaciones con fechas de inicio y fin, **inhabilitando automáticamente su disponibilidad en la agenda online** para proteger la calidad de servicio.

5. El Backend y sus Reglas de Negocio

El backend se estructura mediante controladores de API REST en `src/app/api/`:

- **Algoritmo Anti-Colisión de Horarios (`/api/citas`):** Al recibir una solicitud, el backend calcula la duración real del servicio y verifica en la base de datos si el especialista ya tiene una cita reservada que se solape en ese rango. Si existe conflicto, rechaza la solicitud con código HTTP 409.
- **Control de Inventario Bidireccional (`/api/pedidos`):** Al registrarse una compra en línea, se descuenta de inmediato el stock físico. Si el pedido es cancelado por el administrador, el sistema repone automáticamente las unidades al inventario.
- **Seguridad por Roles (RBAC):** El Middleware de Next.js verifica las credenciales y rol del usuario; si un cliente intenta ingresar a `/admin`, es redirigido al formulario de acceso administrativo.
- **Escritura Atómica Segura:** La función `writeDB()` escribe primero en un archivo temporal y luego realiza un renombramiento atómico en disco, previniendo corrupción de datos ante caídas eléctricas imprevistas.

6. La Joya Tecnológica: ¿Cómo es Posible la Colorimetría con IA?

> **Modelo Utilizado:** `hair_segmenter.tflite` de Google Research (Red Neuronal Convolucional para Segmentación Semántica).
> **Tecnología de Ejecución:** WebAssembly (WASM) nativo y Canvas 2D en el dispositivo del cliente.

El simulador funciona mediante un flujo de visión artificial en 5 etapas:

1. **Redimensión Inteligente:** Si el cliente sube una fotografía en alta resolución (ej. 4K o 12 MP desde un móvil moderno), el sistema la reescala internamente a un máximo de 1024 píxeles manteniendo su proporción original, garantizando respuesta en menos de 1 segundo.
2. **Segmentación de Fibra Capilar:** La red neuronal analiza los píxeles y genera una máscara de probabilidad matemática (`Float32Array`) asignando a cada coordenada un valor de 0.0 (fondo/rostro) a 1.0 (cabello).
3. **Doble Motor de Aceleración (GPU → CPU):** Intenta procesar mediante aceleración por hardware WebGL/GPU; si el dispositivo no posee GPU compatible o está en una laptop básica, conmuta de forma automática y transparente a procesamiento en CPU sin arrojar error.
4. **Cálculo de Luminosidad y Volumen Tridimensional:** En lugar de sobreponer una plasta de color plano, el algoritmo calcula la luminosidad original de cada hebra:

   $$\text{Luminosidad} = 0.299 \times \text{Rojo} + 0.587 \times \text{Verde} + 0.114 \times \text{Azul}$$

   Esto preserva intactas las luces naturales, las sombras y la textura rizada u ondulada.
5. **Formulación Cromática de Salón:** Aplica curvas de transferencia de color para 7 tonos reales (Rubio Miel, Rubio Cenizo, Castaño Caramelo, Chocolate Profundo, Cobrizo Natural, Borgoña Vino y Negro Azabache).

7. Preguntas Difíciles del Jurado y Respuestas Preparadas

P1: ¿Por qué eligieron una base de datos documental JSON y no un gestor relacional como PostgreSQL o MySQL?

Respuesta: "Para el alcance de esta fase en un salón de belleza especializado, una base de datos documental JSON local nos permitió desacoplar la solución de servidores o dependencias pesadas, logrando portabilidad total en cualquier máquina con Node.js sin necesidad de configuraciones externas. Sin embargo, la arquitectura está completamente modularizada en `src/lib/db/index.ts`, por lo que migrar a PostgreSQL o Prisma se realiza cambiando únicamente las funciones de lectura y escritura sin alterar ni la interfaz de usuario ni las rutas del sistema."

P2: ¿Cómo evitan que dos clientes reserven la misma cita al mismo tiempo?

Respuesta: "Implementamos una doble barrera: en el Frontend, el componente del calendario consulta las citas y deshabilita visualmente los bloques de horas ocupados. En el Backend, la API `/api/citas` vuelve a comprobar la disponibilidad en el milisegundo exacto de recibir la petición; si detecta una colisión de horario y especialista, rechaza la operación con código HTTP 409 de conflicto."

P3: ¿Cómo adaptaron la plataforma a la realidad comercial de Venezuela?

Respuesta: "Se diseñó con tres características clave para el contexto local: primero, formato de precios en divisas estandarizado (`X,XX$`); segundo, pasarela manual de Pago Móvil con selector de bancos nacionales (BDV, Banesco, Mercantil, Bancamiga, Provincial, BNC) y registro de número de referencia; y tercero, cobro de anticipo del 20% para asegurar el compromiso de asistencia al salón."

P4: ¿Por qué la colorimetría no usa una API en la nube como Gemini o Stable Diffusion?

Respuesta: "Por tres motivos estratégicos: primero, **costo operativo cero ($0)**, ya que no se consumen créditos ni servidores en la nube; segundo, **privacidad total**, ya que la fotografía de la clienta jamás sale de su dispositivo; y tercero, **velocidad y disponibilidad offline**, permitiendo que la clienta pruebe tonos en menos de un segundo incluso con conexiones de internet lentas."

P5: ¿Qué medidas de optimización de rendimiento se implementaron en el sitio?

Respuesta: "Aplicamos optimizaciones integrales: rediseño de imágenes a formato WebP moderno reduciendo más del 65% de peso; reducción de la fuente de iconos Material Symbols de 3.9 MB a solo 5.8 KB conservando únicamente los 26 glifos utilizados; carga diferida mediante `next/dynamic` para secciones por debajo del pliegue; y caché en memoria para las lecturas de la base de datos."

8. Ejecución con un Solo Clic en Entornos de Demostración

Para facilitar la evaluación y puesta en marcha durante la defensa en cualquier computadora o laptop, se empaquetó el proyecto con scripts de automatización:

- `INICIAR_SISTEMA.bat`: Detecta Node.js, verifica dependencias y binarios WASM, compila en modo optimizado de alta velocidad e inicia el navegador automáticamente en `http://localhost:3000`.
- `Crear_Acceso_Directo_Escritorio.bat`: Coloca el icono de acceso directo en el Escritorio de Windows para iniciar la plataforma con un solo clic.
